



MSA UNIVERSITY
جامعة أكتوبر للعلوم الحديثة والآداب



UNIVERSITY of
GREENWICH

Test Conditions & Cases: Contact Manager System

Software Testing, Validation and Verification (CSE265)

Project: C++ Console-based Contact Manager System

Under the Supervision of: Dr. Gehad Mohey

Version: 1.3

Date: May 10, 2026

Student Name:

ID:

Mazen Yasser

247861

Jasmine Ali

240247

Sandy Atef

245077

Mariam Mohamed Reyad

245171

Marwan Ehab

248889

Table of Contents

Test Conditions & Cases: Contact Manager System	1
1.Introduction	3
2.Unit Testing	3
For ADD Contact:	3
For Search Contact:	5
For Update Contact:	7
For Delete Contact:	8
3.Integration Testing	9
4.Dynamic Testing.....	10
5.System Testing	11
6.Conclusion	11

1.Introduction

Test conditions describe what needs to be tested in the Contact Management System. The test cases are then created based on these conditions to verify whether the system behaves correctly.

2.Unit Testing

For ADD Contact:

Taking requirement FR-AC, we can deduce:

1. Verify that the system adds a contact when the name and phone number are valid.
2. Verify that the system does not add a contact when the name is empty.
3. Verify that the system does not add a contact when the phone number is empty.
4. Verify that the system does not add a duplicate contact.

Deduced Decision Table:

Conditions	1	2	3	4
Name Valid	Y	N	Y	Y
Phone Valid	Y	Y	N	Y
Contact exists	N	N	N	Y
Action				
Add Contact	Y			
Show Error		Y	Y	Y

Deduced Test Cases:

Test Case ID	Requirement Reference	Description	Test Data	Expected Outcome
UT-01	FR-AC-01	Verify that the system adds a contact with valid data.	Name: Ali, Phone: 01012345678	Contact is added successfully.
UT-02	FR-AC-01	Verify that the system does not add a contact with an empty name.	Name: empty, Phone: 01012345678	Add operation fails and an error message is displayed.
UT-03	FR-AC-01	Verify that the system does not add a contact with an empty phone number.	Name: Ali, Phone: empty	Add operation fails and an error message is displayed.
UT-04	FR-AC-02	Verify that the system assigns unique IDs to each contact	3 different contacts with different names and phones	Each contact is assigned a unique ID
UT-05	FR-AC-01	Verify that the phone format is valid	Name: Ali, Phone:abd123	Contact is not added and invalid error is displayed
UT-06	FR-AC-01	Verify that the system does not add a duplicate contact.	Existing contact: Ali, 01012345678 New contact: Ali, 01012345678	Contact is not added and duplicate error is displayed.

For Search Contact:

Taking requirement FR-SC, we can deduce:

1. Verify that the system finds an existing contact.
2. Verify that the system does not find a non-existing contact.
3. Verify that the system handles empty search input.

Deduced Decision Table:

Conditions	1	2	3
Search input valid	Y	Y	N
Contact exists	Y	N	-
Action			
Display contact	✓		
Display not found message		✓	
Display input error			✓

Deduced Test Cases:

Test Case ID	Requirement Reference	Description	Test Data	Expected Outcome
UT-07	FR-SC-01	Verify that the system finds an existing contact by name.	Search name: Ali	Ali's contact details are displayed.
UT-09	FR-SC-01	Verify that the system shows not found for a non-existing contact.	Search name: Omar	System displays "Contact not found."
UT-08	FR-SC-01	Verify that the system searches with partial part of name	Search name: Al	System displays contacts containing Al.

For Update Contact:

Taking requirement FR-UC, we can deduce:

1. Verify that the system updates an existing contact with valid new data.
 2. Verify that the system does not update a non-existing contact.
 3. Verify that the system does not update a contact with an empty name.
 4. Verify that the system does not update a contact with an empty phone number.
- Deduced Test Cases:

Test Case ID	Requirement Reference	Description	Test Data	Expected Outcome
UT-12	FR-UC-01	Verify that the system updates an existing contact with valid data.	Existing: Ali, 01012345678 New phone: 01111111111	Contact is updated successfully.
UT-13	FR-UC-01	Verify that the system does not update a non-existing contact.	Search name: Mona	Update fails and "Contact not found" is displayed.

For Delete Contact:

Taking requirement FR-DC, we can deduce:

1. Verify that the system deletes an existing contact.
2. Verify that the system does not delete a non-existing contact.
3. Verify that the system confirms the deletion.

Deduced Test Cases:

Test Case ID	Requirement Reference	Description	Test Data	Expected Outcome
UT-10	FR-DC-01	Verify that the system deletes an existing contact.	Existing contact ID	Contact is removed from the contact list.
UT-11	FR-DC-01	Verify that the system does not delete a non-existing contact.	Non-Existing contact ID	Delete fails and "Contact not found" is displayed.

3.Integration Testing

Test Case ID	Test Case	Steps	Expected Result
IT-01	Add contact integration	Console → Service → Save JSON	Contact stored in file
IT-02	View contacts integration	Load from JSON → display	All contacts shown correctly
IT-03	Search integration	Input name → service → file data	Correct match returned
IT-04	Update integration	Modify contact → save → reload	Updated data appears in file
IT-05	Delete integration	Remove contact → update file	Contact removed from JSON

4.Dynamic Testing

Test Case ID	Test Case	Input	Expected Result
DT-01	Add contact via menu	1 → name + phone	Contact added successfully
DT-02	View contacts	Menu option 2	All contacts displayed
DT-03	Search contact	Menu option 3 + name	Matching results shown
DT-04	Update contact	Menu option 4 + ID	Data updated successfully
DT-05	Delete contact	Menu option 5 + ID	Contact removed
DT-06	Invalid menu choice	Input: 99	Error message shown
DT-07	Empty input handling	Empty name/phone	Validation error shown
DT-08	Manual JSON modification then update	Menu option 4 + ID + Update	Error message shown
DT-09	Termination during add operation	Menu option 1 → name + Termination	JSON file data persistence

5.System Testing

Test Case ID	Test Case	Steps	Expected Result
ST-01	Full workflow test	Add → View → Search → Update → Delete	All operations work correctly
ST-02	Startup test	Run program	Contacts loaded from JSON or empty list created
ST-03	Data persistence test	Add contact → restart program	Contact still exists
ST-04	End-to-end update test	Add → update → view	Updated data shown correctly
ST-05	End-to-end delete test	Add → delete → view	Contact no longer exists
ST-06	Menu navigation test	Use all menu options	No crashes, correct routing
ST-07	JSON file recovery test	Missing or empty JSON file	Program handles missing or empty JSON file without crashing

6.Conclusion

The test conditions and test cases cover the main functionality of the Contact Management System. Unit test cases focus on validating individual functions in the service layer, while integration test cases verify that the service and repository work together correctly. Dynamic and system test cases ensure that the program behaves correctly during actual execution and that the full user workflow is completed successfully.